

راهنمای مرجع

# تست‌نویسی در React

مفاهیم پایه، انواع تست، مثال‌های عملی و قوانین طلایی برای نوشتن  
تهیه شده برای رامین — یک منبع مرجع شخصی

تست‌های حرفه‌ای با React + TypeScript + Vite + Vitest

React • TypeScript • Vite • Vitest • Testing Library



## فهرست مطالب

۱. هدف تست‌نویسی چیست و چرا اهمیت دارد
۲. انواع تست‌نویسی (Unit، Integration، E2E و بیشتر)
۳. هرم تست (Testing Pyramid)
۴. ابزارهای اصلی در اکوسیستم React
۵. مثال عملی: تست یک تابع ساده (Unit)
۶. مثال عملی: تست رندر کامپوننت UI
۷. اولویت‌بندی Query ها در Testing Library
۸. مثال عملی: تست فرم و تعامل کاربر
۹. مثال عملی: Mock کردن توابع با vi.fn
۱۰. مثال عملی: تست async و درخواست‌های API
۱۱. مثال عملی: تست کامپوننت با Context/Provider
۱۲. مثال عملی: تست یک Custom Hook
۱۳. قوانین طلایی تست‌نویسی
۱۴. چک‌لیست نهایی قبل از commit
۱۵. MSW برای mock کردن API
۱۶. vi.mock برای mock کردن یک module

- ۱۷. کنترل زمان با Fake Timers
- ۱۸. تست‌های پارامتری با it.each
- ۱۹. تست کامپوننت با React Router
- ۲۰. تست کامپوننت متصل به Zustand/Redux
- ۲۱. مثال عملی: تست یک فلوی کامل (Integration واقعی)
- ۲۲. سازماندهی پروژه و فایل‌ها

## ۱. هدف تست‌نویسی چیست و چرا اهمیت دارد

تست‌نویسی یعنی نوشتن کدی که به صورت خودکار بررسی می‌کند آیا بخش دیگری از کد، همان‌طور که انتظار داریم رفتار می‌کند یا نه. بدون تست، تنها راه اطمینان از درست کار کردن برنامه، تست دستی در مرورگر است که هم زمان‌بر است و هم قابل اعتماد نیست.

### چرا تست می‌نویسیم؟

- **جلوگیری از Regression:** مطمئن می‌شویم تغییرات جدید، چیزی که قبلاً درست کار می‌کرد را خراب نکرده.
- **Refactor با اطمینان:** می‌توان ساختار کد را بازنویسی کرد بدون ترس از خراب شدن رفتار.
- **مستندسازی زنده:** تست‌ها نشان می‌دهند یک کامپوننت یا تابع دقیقاً باید چه رفتاری داشته باشد.
- **کشف باگ زودتر:** باگ‌ها قبل از رسیدن به کاربر واقعی و در همان مرحله توسعه پیدا می‌شوند.
- **اعتماد تیمی:** در تیم‌های بزرگ، تست‌ها تضمین می‌کنند کد یک نفر، کد نفر دیگر را خراب نمی‌کند.

## ۲. انواع تست نویسی

| نوع تست           | چه چیزی را بررسی می کند                                            | مثال در React                                                       | سرعت اجرا |
|-------------------|--------------------------------------------------------------------|---------------------------------------------------------------------|-----------|
| Unit Test         | یک واحد کوچک و مجزا از کد (تابع، هوک، کامپوننت ساده)               | تست تابع <code>sum()</code> یا یک کامپوننت <code>Button</code>      | خیلی سریع |
| Integration Test  | چند واحد که با هم ترکیب شده اند و باید هماهنگ کار کنند             | فرمی با چند <code>input</code> که با هم <code>submit</code> می شوند | سریع      |
| End-to-End (E2E)  | کل برنامه از دید یک کاربر واقعی، در مرورگر واقعی                   | باز کردن سایت، لاگین، خرید محصول (با Playwright/Cypress)            | کند       |
| Snapshot Test     | مقایسه خروجی رندر شده با یک نسخه ذخیره شده قبلی                    | تشخیص تغییرات ناخواسته در ساختار UI                                 | سریع      |
| Static Type Check | بررسی صحت تایپ ها قبل از اجرای برنامه (نه دقیقاً تست، ولی مکمل آن) | خطاهای TypeScript در زمان کامپایل                                   | آنی       |

### نکته

در پروژه های React معمولی، بیشترین ارزش از **Unit** و **Integration** تست ها به دست می آید، چون هم سریع اجرا می شوند و هم مستقیماً با ابزارهایی مثل Vitest و Testing Library نوشته می شوند. E2E برای سناریوهای حیاتی و مسیرهای اصلی کاربر (مثل فرآیند خرید) نگه داشته می شود، چون نگهداری اش پرهزینه تر است.

## ۳. هرم تست (Testing Pyramid)

یک راهنمای کلی برای تصمیم‌گیری در مورد اینکه «چقدر» از هر نوع تست بنویسیم:

E2E — تعداد کم، سناریوهای حیاتی

Integration — تعداد متوسط

Unit — بیشترین تعداد، سریع و ارزان

هرچه به سمت بالای هرم برویم، تست‌ها واقعی‌تر ولی کندتر و شکننده‌تر می‌شوند. هرچه به سمت پایین برویم، تست‌ها سریع‌تر و ارزان‌تر ولی محدودتر در پوشش سناریوهای واقعی هستند. تعادل درست معمولاً یعنی نوشتن تست‌های زیاد Unit، مقدار معقولی Integration، و تعداد کمی E2E روی مسیرهای حیاتی.

## ۴. ابزارهای اصلی در اکوسیستم React

| ابزار                                          | نقش                                                                                 |
|------------------------------------------------|-------------------------------------------------------------------------------------|
| <code>vitest</code>                            | اجراکننده تست ((test runner, جایگزین سریع Jest که با Vite هماهنگ است                |
| <code>@testing-library/react</code>            | رندر کردن کامپوننت‌های React در محیط تست و دسترسی به DOM آن‌ها                      |
| <code>@testing-library/user-event</code>       | شبیه‌سازی واقعی‌تر تعاملات کاربر (تایپ، کلیک، انتخاب)                               |
| <code>@testing-library/jest-dom</code>         | matcher های اضافه مثل <code>toBeInTheDocument()</code> , <code>toHaveValue()</code> |
| <code>jsdom</code>                             | شبیه‌سازی DOM مرورگر درون Node.js برای اجرای تست‌ها بدون مرورگر واقعی               |
| <code>msw</code> (Mock Service Worker))        | استاندارد صنعتی برای mock کردن درخواست‌های شبکه در سطح شبکه، نه دستی                |
| <code>Cypress</code> / <code>Playwright</code> | اجرای تست‌های E2E در یک مرورگر واقعی                                                |



## ۵. مثال عملی: تست یک تابع ساده (Unit))

کد اصلی — `src/utils/math.ts`

```
export function sum(a: number, b: number): number {
  return a + b
}

export function isEven(num: number): boolean {
  return num % 2 === 0
}
```

فایل تست — `src/utils/math.test.ts`

```
import { describe, it, expect } from 'vitest'
import { sum, isEven } from './math'

describe('math utils', () => {
  it('should add two numbers correctly', () => {
    expect(sum(2, 3)).toBe(5)
  })

  it('should return true for even numbers', () => {
    expect(isEven(4)).toBe(true)
  })
})
```

الگوی AAA

هر تست خوب سه بخش دارد: **Arrange** (آماده سازی داده)، **Act** (اجرای عملیات)، **Assert** (بررسی نتیجه).

## ۶. مثال عملی: تست رندر کامپوننت UI

### کامپوننت — src/components/Greeting.tsx

```
type GreetingProps = { name: string }

export function Greeting({ name }: GreetingProps) {
  return (
    <div>
      <h1>سلام، {name}!</h1>
      <p>به اپلیکیشن ما خوش اومدی</p>
    </div>
  )
}
```

### تست — src/components/Greeting.test.tsx

```
import { render, screen } from '@testing-library/react'
import { Greeting } from './Greeting'

describe('Greeting component', () => {
  it('should render the name passed as prop', () => {
    render(<Greeting name="علی" />)

    expect(screen.getByText('!سلام ، علی')).toBeInTheDocument()
  })
})
```

## فلسفه اصلی Testing Library

"Test the way your users use your app" — یعنی به جزئیات پیاده‌سازی داخلی (implementation details) کاری نداشته باش، فقط چیزی را تست کن که کاربر واقعی می‌بیند و با آن تعامل دارد.

## ۷. اولویت‌بندی Query ها در Testing Library

| اولویت        | Query                                                      | چه زمانی استفاده شود                                               |
|---------------|------------------------------------------------------------|--------------------------------------------------------------------|
| ۱ (بهترین)    | <code>getByRole</code>                                     | دکمه، هدینگ، لینک، فیلد ورودی — همان چیزی که screen reader می‌بیند |
| ۲             | <code>getByLabelText</code>                                | فیلدهای فرم که label دارند                                         |
| ۳             | <code>getByText</code> / <code>getByPlaceholderText</code> | وقتی گزینه بهتری در دسترس نیست                                     |
| ۴ (آخرین راه) | <code>getByTestId</code>                                   | فقط زمانی که هیچ‌کدام از موارد بالا جواب نمی‌دهد                   |

### نقش‌های پیش‌فرض تگ‌های HTML

| تگ HTML                                            | Role پیش‌فرض         |
|----------------------------------------------------|----------------------|
| <code>&lt;button&gt;</code>                        | <code>button</code>  |
| <code>&lt;a href=“...”&gt;</code>                  | <code>link</code>    |
| <code>&lt;h1&gt;</code> تا <code>&lt;h6&gt;</code> | <code>heading</code> |
| <code>&lt;img&gt;</code>                           | <code>img</code>     |
| <code>&lt;ol&gt;</code> / <code>&lt;ul&gt;</code>  | <code>list</code>    |
| <code>&lt;input type=“text”&gt;</code>             | <code>textbox</code> |

## ابزار دیباگ

اگر مطمئن نیستید چه role هایی در صفحه رندر شده وجود دارد: `screen.debug()` کل HTML را در ترمینال چاپ می‌کند.

## ۸. مثال عملی: تست فرم و تعامل کاربر

```
import userEvent from '@testing-library/user-event'
import { render, screen } from '@testing-library/react'
import { LoginForm } from './LoginForm'

it('should call onSubmit with correct values', async () => {
  const user = userEvent.setup()
  const handleSubmit = vi.fn()

  render(<LoginForm onSubmit={handleSubmit} />)

  await user.type(screen.getByLabelText('ایمیل'), 'ali@example.com')
  await user.type(screen.getByLabelText('رمز عبور'), '123456')
  await user.click(screen.getByRole('button', { name: 'ورود' }))

  expect(handleSubmit).toHaveBeenCalledWith('ali@example.com', '123456')
})
```

### نکته مهم

همیشه قبل از هر تست `userEvent.setup()` صدا زده می‌شود، و هر عملیات تعاملی (type, click) باید با `await` فراخوانی شود چون به صورت `async` شبیه‌سازی می‌شود.

## ۹. مثال عملی: Mock کردن توابع با vi.fn

```
const handleClick = vi.fn()

render(<Button onClick={handleClick}>کلیک کن</Button>)
await userEvent.click(screen.getByRole('button'))

expect(handleClick).toHaveBeenCalledTimes(1)
expect(handleClick).toHaveBeenCalledWith(/* آرگومان های مورد انتظار */)

```

یک تابع «جاسوسی» (spy) می‌سازد که رفتار خودش را ندارد، ولی ثبت می‌کند چند بار و با چه آرگومان‌هایی صدا زده شده است. این برای تست کردن ارتباط بین یک کامپوننت فرزند و parent آن، بدون نیاز به اجرای منطق واقعی، بسیار کاربردی است.



## ۱۰. مثال عملی: تست async و درخواست‌های API

```
beforeEach(() => {
  vi.stubGlobal('fetch', vi.fn())
})

afterEach(() => {
  vi.unstubAllGlobals()
})

it('should display user data after successful fetch', async () => {
  vi.mocked(fetch).mockResolvedValue({
    ok: true,
    json: async () => ({ id: 1, name: 'رامین' }),
  } as Response)

  render(<UserProfile userId={1} />)

  // findBy می‌رسد async چون داده به‌صورت
  const name = await screen.findByText('رامین')
  expect(name).toBeInTheDocument()
})
```

| Query                    | رفتار                                                    |
|--------------------------|----------------------------------------------------------|
| <code>getByText</code>   | فوری چک می‌کند، اگر نبود بلافاصله fail می‌شود            |
| <code>findByText</code>  | async است، چند بار retry می‌کند — مناسب برای نتیجه async |
| <code>queryByText</code> | اگر نبود null برمی‌گرداند — برای بررسی «عدم وجود»        |

## جایگزین بهتر: MSW

به جای mock دستی `fetch` ، ابزار **Mock Service Worker** (`msw`) در سطح شبکه درخواست‌ها را می‌گیرد و پاسخ فیک برمی‌گرداند — به کد واقعی نزدیک‌تر و قابل‌نگهداری‌تر است.

## ۱۱. مثال عملی: تست کامپوننت با Context/Provider

اگر کامپوننتی از `useContext` یا از `react-router` استفاده کند، هنگام `render` آن باید `Provider` مربوطه را هم دور آن قرار دهیم، وگرنه با خطا مواجه می‌شویم.

```
function renderWithTheme(ui: React.ReactElement) {
  return render(
    <ThemeProvider theme={'dark'}>
      {ui}
    </ThemeProvider>
  )
}

it('should render with the dark theme applied', () => {
  renderWithTheme(<Header />)
  expect(screen.getByRole('banner')).toHaveClass('dark')
})
```

یک الگوی رایج برای تمیز نگه داشتن تست‌ها، ساخت یک تابع کمکی `renderWithProviders` است که همه `Provider` های لازم (Theme، Router، Redux Store ...) را یکجا اضافه می‌کند تا در هر تست تکرار نشود.

## ۱۲. مثال عملی: تست یک Custom Hook

برای هوک‌هایی که مستقل از یک کامپوننت خاص هستند، به جای رندر کردن یک کامپوننت کامل، از `renderHook` استفاده می‌کنیم.

```
import { renderHook, act } from '@testing-library/react'
import { useCounter } from './useCounter'

it('should increment the counter', () => {
  const { result } = renderHook(() => useCounter())

  act(() => {
    result.current.increment()
  })

  expect(result.current.count).toBe(1)
})
```

### چرا `act` لازم است؟

وقتی داخل تست، مستقیماً متدی را صدا می‌زنیم که باعث تغییر `state` می‌شود، باید آن را داخل `act()` بپیچیم تا React مطمئن شود همه به‌روزرسانی‌های مربوطه قبل از assertion بعدی اعمال شده‌اند.

## ۱۳. قوانین طلایی تست‌نویسی

### ۱. رفتار را تست کن، نه پیاده‌سازی را

به state داخلی، نام متغیرها یا جزئیات پیاده‌سازی وابسته نشو؛ فقط چیزی را تست کن که کاربر می‌بیند.

### ۲. اولویت با `getByRole` است

اگر همیشه مجبوری از `getById` استفاده کنی، احتمالاً یعنی کامپوننت از نظر accessibility ضعیف است.

### ۳. اسم تست باید مثل مستندات خوانده شود

اگر تستی fail شد، باید فقط از روی اسمش دقیقاً بفهمی چه رفتاری خراب شده.

### ۴. هر تست باید مستقل باشد

تست‌ها نباید به ترتیب اجرا یا نتیجه‌ی تست دیگر وابسته باشند.

### ۵. `Mock` را کم و هدفمند نگه دار

فقط چیزهای خارج از کنترل تست (شبکه، زمان، API مرورگر) را mock کن، نه منطق خود کامپوننت.

### ۶. هم مسیر موفق و هم مسیر خطا را تست کن

تست کردن فقط happy path کافی نیست؛ حالت‌های خطا و edge case ها اغلب جایی هستند که باگ واقعی رخ می‌دهد.

## ۷. Coverage عدد بالا هدف نیست

رسیدن به coverage صد درصد به معنی نبود باگ نیست؛ روی مسیرهای پرخطر و منطق پیچیده تمرکز کن.

## اشتباه رایج

تست کردن جزئیات پیاده‌سازی داخلی، مثل: `expect(component.state.isOpen).toBe(true)` — این نوع تست با هر تغییر داخلی کد می‌شکند، حتی اگر رفتار واقعی درست باشد.

## ۱۴. چک لیست نهایی قبل از commit

- اسم تست ها گویا و خوانا هستند
- از getByRole به جای getByTestId استفاده شده (در صورت امکان)
- تست ها به ترتیب اجرا یا به هم وابسته نیستند
- mock ها بعد از هر تست پاک/ریست می شوند (afterEach)
- هم حالت موفق (happy path) و هم حالت خطا تست شده
- برای عملیات async از await و findBy/waitFor استفاده شده
- هر Provider لازم (Theme/Router/Store) هنگام render اضافه شده

## ۱۵. MSW برای mock کردن API

MSW (Mock Service Worker) استاندارد صنعتی برای mock کردن درخواست‌هاست — برخلاف mock دستی fetch، درخواست‌ها را در سطح شبکه می‌گیرد، یعنی به کد واقعی نزدیک‌تر است.

### نصب

```
npm install -D msw
```

### تعریف handler ها — src/mocks/handlers.ts

```
import { http, HttpResponse } from 'msw'

export const handlers = [
  http.get('/api/users/:id', ({ params }) => {
    const { id } = params

    if (id === '999') {
      return new HttpResponse(null, { status: 404 })
    }

    return HttpResponse.json({
      id: Number(id),
      name: 'رامین',
      email: 'ramin@example.com',
    })
  }),
]
```



## src/mocks/server.ts — راه اندازی سرور برای تست

---

```
import { setupServer } from 'msw/node'
import { handlers } from '../handlers'

export const server = setupServer(...handlers)
```

## src/test/setup.ts در فعال سازی

---

```
import '@testing-library/jest-dom'
import { server } from '../mocks/server'

beforeAll(() => server.listen())
afterEach(() => server.resetHandlers())
afterAll(() => server.close())
```

## تست تمیز و بدون vi.stubGlobal

---

```
it('should display user data after successful fetch', async () => {
  render(<UserProfile userId={1} />)
  expect(await screen.findByText('رامین')).toBeInTheDocument()
})

it('should show error for a non-existent user', async () => {
  render(<UserProfile userId={999} />)
  expect(await screen.findByRole('alert')).toHaveTextContent('کاربر پیدا نشد')
})
```

| مزیت               | توضیح                                                    |
|--------------------|----------------------------------------------------------|
| نزدیک‌تر به واقعیت | درخواست واقعی در سطح شبکه گرفته می‌شود، نه جایگزینی تابع |
| قابل استفاده مجدد  | همان handler ها در تست و در dev mode قابل استفاده‌اند    |
| تمیزتر             | نیازی به beforeEach/afterEach دستی برای fetch نیست       |
| مستقل از کتابخانه  | چه fetch چه axios، فرقی نمی‌کند                          |

### Override کردن در یک تست خاص

```
import { server } from '../mocks/server'
import { http, HttpResponse } from 'msw'

it('should handle server error', async () => {
  server.use(
    http.get('/api/users/:id', () => new HttpResponse(null, { status: 500 }))
  )

  render(<UserProfile userId={1} />)
  expect(await screen.findByRole('alert')).toBeInTheDocument()
})
```

## ۱۶. vi.mock برای mock کردن یک module

گاهی باید یک module کامل local (نه یک درخواست شبکه) را جایگزین کنیم — مثلاً یک فایل helper که با یک SDK خارجی کار می‌کند.

### کد اصلی — src/lib/analytics.ts

```
export function trackEvent(name: string, payload: object) {  
  window.analytics?.track(name, payload)  
}
```

### کامپوننت — src/components/BuyButton.tsx

```
import { trackEvent } from '../lib/analytics'  
  
export function BuyButton() {  
  function handleClick() {  
    trackEvent('buy_clicked', { source: 'homepage' })  
  }  
  return <button onClick={handleClick}>خرید</button>  
}
```

### تست

```
import { trackEvent } from '../lib/analytics'  
  
// می‌شوند vi.fn() هایش خودکار export می‌کنیم؛ همه mock کل ماژول را  
vi.mock('../lib/analytics')
```

```
it('should track event when buy button is clicked', async () => {
  const user = userEvent.setup()
  render(<BuyButton />)

  await user.click(screen.getByRole('button', { name: 'خرید' }))

  expect(trackEvent).toHaveBeenCalledWith('buy_clicked', { source: 'homepage' })
})
```

vi.mock باید hoisted باشد

Vitest به صورت خودکار فراخوانی vi.mock(...) را به بالای فایل منتقل می کند (قبل از import ها اجرا می شود)، ولی بهتر است همیشه بالای فایل نوشته شود تا خوانا بماند.

| ابزار   | چه زمانی استفاده شود                                                                  |
|---------|---------------------------------------------------------------------------------------|
| MSW     | وقتی چیزی که mock می کنید یک درخواست شبکه (HTTP) است                                  |
| vi.mock | وقتی چیزی که mock می کنید یک ماژول/فایل local است (SDK، helper، کامپوننت فرزند سنگین) |

## ۱۷. کنترل زمان با Fake Timers

کامپوننت — `src/components/Toast.tsx`

```
export function Toast({ message }: { message: string }) {
  const [visible, setVisible] = useState(true)

  useEffect(() => {
    const timer = setTimeout(() => setVisible(false), 3000)
    return () => clearTimeout(timer)
  }, [])

  if (!visible) return null
  return <div role="status">{message}</div>
}
```

بدون fake timer، باید واقعاً ۳ ثانیه در تست صبر کنید. با fake timers می‌توانید زمان را جلو ببرید:

```
import { act } from '@testing-library/react'

describe('Toast component', () => {
  beforeEach(() => {
    vi.useFakeTimers()
  })

  afterEach(() => {
    vi.useRealTimers()
  })

  it('should disappear after 3 seconds', () => {
    render(<Toast message="ذخیره شد" />)
  })
})
```

```
expect(screen.getByRole('status')).toBeInTheDocument()

// زمان را ۳ ثانیه جلو می‌بریم، بدون صبر واقعی
act(() => {
  vi.advanceTimersByTime(3000)
})

expect(screen.queryByRole('status')).not.toBeInTheDocument()
})
})
```

### چرا `act()` اینجا لازم است؟

اجرای `setTimeout` توسط `advanceTimersByTime` باعث صدا زده شدن `setVisible(false)` بیرون از یک event handler عادی React می‌شود. بدون `act()`، تضمینی نیست که `render` ناشی از این تغییر state قبل از خط `expect` بعدی روی DOM اعمال شده باشد — نتیجه‌اش خطایی شبیه `expected document not to contain element, found <div role="status">` است، حتی وقتی منطق کامپوننت کاملاً درست است.

### ترکیب با `userEvent`

اگر در همان تست هم از `userEvent` و هم از `vi.useFakeTimers()` استفاده می‌کنید، باید موقع `setup` این را هم مشخص کنید، وگرنه `await user.click(...)` ممکن است گیر کند:

```
const user = userEvent.setup({ advanceTimers: vi.advanceTimersByTime })
```

### کنترل تاریخ سیستم

اگر کامپوننت از `Date.now()` هم استفاده کند، با `vi.setSystemTime(new Date('2026-01-01'))` قابل کنترل است.

## ۱۸. تست‌های پارامتری با `it.each`

وقتی چند ورودی مختلف همان منطق را تست می‌کنند، به جای کپی-پیست از `it.each` استفاده کنید:

```
describe('isEven', () => {
  it.each([
    [2, true],
    [3, false],
    [0, true],
    [-4, true],
    [-7, false],
  ])('isEven(%i) should return %s', (input, expected) => {
    expect(isEven(input)).toBe(expected)
  })
})
```

خروجی این‌طور می‌شود، هر ردیف یک تست جدا حساب می‌شود:

```
✓ isEven(2) should return true
✓ isEven(3) should return false
✓ isEven(0) should return true
✓ isEven(-4) should return true
✓ isEven(-7) should return false
```

برای `object` های پیچیده‌تر، از یک آرایه از `object` ها استفاده کنید:

```
it.each([
  { email: '', password: '123456', expectedError: 'ایمیل الزامی است' },
  { email: 'a@b.com', password: '', expectedError: 'رمز عبور الزامی است' },
])('should show "$expectedError" when email is "$email"', async ({ email, password, expectedError }) => {
  // ...
})
```

```
expect(screen.getByRole('alert')).toHaveTextContent(expectedError)
})
```

چرا مفید است؟

از تکرار کد جلوگیری می‌کند، مخصوصاً برای validation های فرم که چندین حالت خطا دارند.



## ۱۹. تست کامپوننت با React Router

### کامپوننت — src/pages/ProductPage.tsx

```
import { Link, useNavigate, useParams } from 'react-router-dom'

export function ProductPage() {
  const { productId } = useParams()
  const navigate = useNavigate()

  return (
    <div>
      <h1>محصول شماره {productId}</h1>
      <Link to="/cart">رفتن به سبد خرید</Link>
      <button onClick={() => navigate('/checkout')}>ادامه خرید</button>
    </div>
  )
}
```

چون از `useParams` و `useNavigate` استفاده می‌کند، باید داخل یک `Router` رندر شود. بهترین گزینه `MemoryRouter` است (یک `router` که در حافظه کار می‌کند، نه در مرورگر واقعی):

```
import { MemoryRouter, Routes, Route } from 'react-router-dom'

function renderWithRouter(initialPath: string) {
  return render(
    <MemoryRouter initialEntries={[initialPath]}>
      <Routes>
        <Route path="/products/:productId" element={<ProductPage />} />
      </Routes>
    </MemoryRouter>
  )
}
```

```
it('should display the correct product id from the URL', () => {
  renderWithRouter('/products/42')
  expect(screen.getByText('42 محصول شماره')).toBeInTheDocument()
})

it('should have a link to the cart page', () => {
  renderWithRouter('/products/42')
  const link = screen.getByRole('link', { name: 'رفتن به سبد خرید' })
  expect(link).toHaveAttribute('href', '/cart')
})
```

## تست navigate شدن

```
it('should navigate to checkout when button is clicked', async () => {
  const user = userEvent.setup()

  render(
    <MemoryRouter initialEntries={['/products/42']}>
      <Routes>
        <Route path="/products/:productId" element={<ProductPage />} />
        <Route path="/checkout" element={<h1>صفحه تسویه حساب</h1>} />
      </Routes>
    </MemoryRouter>
  )

  await user.click(screen.getByRole('button', { name: 'ادامه خرید' }))

  // چون صفحه واقعاً عوض شده، محتوای صفحه جدید دیده میشود
  expect(await screen.findByText('صفحه تسویه حساب')).toBeInTheDocument()
})
```

به جای چک کردن اینکه `navigate` با چه آرگومانی صدا زده شده (`implementation detail`)، چک می‌کنیم صفحه واقعاً عوض شده — همان فلسفه‌ی «تست رفتار، نه پیاده‌سازی».

## ۲۰. تست کامپوننت متصل به Zustand/Redux

با Zustand (ساده‌تر، چون فقط یک hook است)

```
// src/store/cartStore.ts
import { create } from 'zustand'

type CartStore = {
  items: string[]
  addItem: (item: string) => void
}

export const useCartStore = create<CartStore>((set) => ({
  items: [],
  addItem: (item) => set((state) => ({ items: [...state.items, item] })),
}))
```

چون Zustand یک store سراسری (global singleton) است، باید state را بین تست‌ها ریست کنید:

```
beforeEach(() => {
  // قبل از هر تست، وگرنه تست‌ها روی هم اثر می‌گذارند
  useCartStore.setState({ items: [] })
})

it('should add item to cart when button is clicked', async () => {
  const user = userEvent.setup()
  render(<AddToCartButton productName="کفش" />)

  await user.click(screen.getByRole('button', { name: 'افزودن به سبد' }))
})
```

```
expect(useCartStore.getState().items).toContain('کفش')
})
```

## با Redux Toolkit (نیاز به Provider دارد)

```
import { Provider } from 'react-redux'
import { configureStore } from '@reduxjs/toolkit'
import cartReducer from '../store/cartSlice'

function renderWithStore(ui: React.ReactElement, preloadedState = {}) {
  const store = configureStore({
    reducer: { cart: cartReducer },
    preloadedState,
  })
  return render(<Provider store={store}>{ui}</Provider>)
}

it('should display the number of items in cart', () => {
  renderWithStore(<CartSummary />, {
    cart: { items: [ 'کیف' , 'کفش' ] },
  })

  expect(screen.getByText('۲ کالا')).toBeInTheDocument()
})
```

### تفاوت کلیدی

با `preloadedState` ، هر تست یک store تازه و ایزوله با state دلخواه می‌سازد — نیازی به ریست دستی نیست، برخلاف Zustand که store سراسری است.

## ۳۱. مثال عملی: تست یک فلوی کامل (Integration واقعی)

یک صفحه لیست محصولات که با کلیک روی «افزودن به سبد»، هم مقدار سبد آپدیت می‌شود هم پیام موفقیت نشان داده می‌شود:

```
// ProductListPage.tsx (خلاصه شده)
export function ProductListPage() {
  return (
    <div>
      <ProductList />
      <CartIndicator />
    </div>
  )
}

function renderPage() {
  const store = configureStore({ reducer: { cart: cartReducer } })
  return render(
    <Provider store={store}>
      <ProductListPage />
    </Provider>
  )
}

it('should update cart indicator and show toast after adding a product', async () => {
  const user = userEvent.setup()
  renderPage()

  // قبل از هر عملی، سبد خالی است
  expect(screen.getByText('۰ کالا در سبد')).toBeInTheDocument()

  // کاربر روی دکمه افزودن اولین محصول کلیک می‌کند
  const addButtons = screen.getAllByRole('button', { name: 'افزودن به سبد' })
  await user.click(addButtons[0])
}
```

```
// هم شمارنده سید آپدیت شده
expect(screen.getByText('کالا در سید')).toBeInTheDocument()

// هم پیام موفقیت نشان داده شده...
expect(await screen.findByRole('status')).toHaveTextContent('به سید اضافه شد')
})
```

### چرا این تست ارزشمند است

برخلاف unit test هایی که هرکدام یک کامپوننت را جدا تست می کنند، این تست تضمین می کند که ارتباط واقعی بین چند کامپوننت (لیست محصول ← store ← نشانگر سید ← toast) درست کار می کند. خیلی از باگ های واقعی production دقیقاً در همین اتصال بین قطعات رخ می دهند، نه داخل یک کامپوننت تنها.

## ۲۲. سازماندهی پروژه و فایل‌ها

الگوی رایج: فایل تست کنار فایل اصلی (co-location)

```
src/  
  components/  
    Button.tsx  
    Button.test.tsx  
    LoginForm.tsx  
    LoginForm.test.tsx  
  hooks/  
    useCounter.ts  
    useCounter.test.ts  
  store/  
    cartSlice.ts  
    cartSlice.test.ts  
  test/  
    setup.ts  
    test-utils.tsx      // توابع کمکی مشترک  
  mocks/  
    handlers.ts  
    server.ts
```

فایل `test-utils.tsx` مرکزی (جلوی تکرار کد را می‌گیرد)

```
// src/test/test-utils.tsx  
import { render, RenderOptions } from '@testing-library/react'  
import { ReactElement } from 'react'  
import { Provider } from 'react-redux'  
import { MemoryRouter } from 'react-router-dom'
```



```
import { configureStore } from '@reduxjs/toolkit'
import cartReducer from '../store/cartSlice'

function renderWithProviders(
  ui: ReactElement,
  { preloadedState = {}, route = '/' }: { preloadedState?: object; route?: string } = {}
) {
  const store = configureStore({ reducer: { cart: cartReducer }, preloadedState })

  return render(
    <Provider store={store}>
      <MemoryRouter initialEntries={[route]}>{ui}</MemoryRouter>
    </Provider>
  )
}

// می‌کنیم export را دوباره testing-library خروجی‌های
export * from '@testing-library/react'
export { renderWithProviders as render }
```

حالا در هر تست:

```
import { render, screen } from '../test/test-utils'
// را دستی بچینید Provider/Router دیگر نیازی نیست خودتان
```

### قوانین سازماندهی

- تست‌ها همیشه کنار فایل اصلی باشند (co-location) — راحت‌تر پیدا می‌شوند.
- منطق مشترک تست (render helper، mock server) در یک پوشه `test/` مرکزی نگه داشته شود.
- اسم فایل تست همیشه یکسان باشد: `File.test.tsx` یا `File.spec.tsx`.

جمع‌بندی این بخش

| ابزار کلیدی                                      | موضوع                       |
|--------------------------------------------------|-----------------------------|
| setupServer + msw                                | Mock حرفه‌ای API            |
| vi.mock('./path')                                | Mock کردن یک ماژول          |
| vi.useFakeTimers(), vi.advanceTimersByTime()     | کنترل زمان                  |
| it.each([...])                                   | کم کردن تکرار تست           |
| MemoryRouter                                     | تست صفحات با Router         |
| preloadedState + Provider یا setState مستقیم     | تست کامپوننت با state مشترک |
| چند کامپوننت با هم، بدون mock کردن ارتباط بینشان | تست فلوی کامل               |
| test-utils.tsx + co-location مرکزی               | سازماندهی                   |

راهنمای مرجع تست‌نویسی React — تهیه شده با Claude